

Tab 1

```
import numpy as np
```

```
PHI_INV = 0.6180339887498948 # 1/phi
```

```
def orthonormalize(v):
```

```
    v = np.asarray(v, dtype=np.float32)
```

```
    v = v / (np.linalg.norm(v) + 1e-12)
```

```
    return v
```

```
def make_rotation_plane(u, v, theta_rad):
```

```
    """
```

```
    Returns a function that rotates a vector h in the 2D plane spanned by u, v  
    by angle theta_rad, leaving all orthogonal components unchanged.
```

```
    """
```

```
    u = orthonormalize(u)
```

```
    v = orthonormalize(v - np.dot(v, u) * u) # Gram-Schmidt
```

```
    cos_t = np.cos(theta_rad)
```

```
    sin_t = np.sin(theta_rad)
```

```
    def rotate(h):
```

```
        h = np.asarray(h, dtype=np.float32)
```

```
        a = np.dot(h, u)
```

```
        b = np.dot(h, v)
```

```
        a_p = cos_t * a - sin_t * b
```

```
        b_p = sin_t * a + cos_t * b
```

```
        # remove old components, add rotated ones
```

```
        h_rot = h - a * u - b * v + a_p * u + b_p * v
```

```
        return h_rot
```

```
    return rotate
```

Tab 2

```

class Rectifier:
    def __init__(self, W_shear=None, strength=0.1):
        """
        W_shear: optional (d x d) matrix defining shear directions.
        If None, use identity (no-op except for potential future tuning).
        strength: how strongly to apply shear.
        """
        self.W = W_shear
        self.strength = strength

    def __call__(self, h):
        h = np.asarray(h, dtype=np.float32)
        if self.W is None:
            return h # placeholder: pure pass-through
        # shear: h' = h + strength * W h
        return h + self.strength * (self.W @ h)

```

Tab 3

```
class Verifier:
    def __init__(self, max_norm=None):
        self.max_norm = max_norm

    def check(self, h):
        h = np.asarray(h, dtype=np.float32)
        if self.max_norm is not None:
            if np.linalg.norm(h) > self.max_norm:
                return False
        return True
```

Tab 4

```

class DarkState(Exception):
    """Raised when invariants fail."""

class GoldenGnomonLayer:
    def __init__(self, dim, seed=0, max_norm=None):
        self.dim = dim
        rng = np.random.default_rng(seed)

        # Choose two random directions for the 108° plane
        u = rng.normal(size=(dim,))
        v = rng.normal(size=(dim,))
        theta = 3.0 * np.pi / 5.0 # 108 degrees

        self.rotate_plane = make_rotation_plane(u, v, theta)
        self.rectifier = Rectifier(W_shear=None, strength=0.0) # start as identity
        self.verifier = Verifier(max_norm=max_norm)

    def stop(self, h):
        # In your runtime, this could snapshot state; here it's just identity.
        return np.asarray(h, dtype=np.float32)

    def rectify(self, h):
        return self.rectifier(h)

    def verify(self, h):
        if not self.verifier.check(h):
            raise DarkState("Invariant violation: norm out of bounds.")
        return h

    def act(self, h):
        # 108° rotation
        h_rot = self.rotate_plane(h)
        # scale by  $\phi^{-1}$ 
        h_scaled = PHI_INV * h_rot
        return h_scaled

    def __call__(self, h):
        # SANTOS: Stop -> Rectify -> Verify -> Act -> Output
        h0 = self.stop(h)
        h1 = self.rectify(h0)
        h2 = self.verify(h1)
        h3 = self.act(h2)
        return h3

```


Tab 5

```
gg = GoldenGnomonLayer(dim=d_model, seed=42, max_norm=10.0)
```

```
h_next = gg(h_current)
```

```
def apply_gg_to_sequence(H, gg_layer):
```

```
    # H: shape (T, d)
```

```
    return np.stack([gg_layer(h_t) for h_t in H], axis=0)
```